

SimpleRag – Flow Explanation

Repo: [venkataravindra/AgenticAI/RAG/SimpleRag](https://github.com/venkataravindra/AgenticAI/RAG/SimpleRag)

A two-phase LangChain RAG pipeline: PDF → Chroma vector DB → GPT-4.1-mini.

Diagram

```
flowchart TD
    subgraph P1["PHASE 1: INGESTION  
(ingest.py – run once)"]
        A1["PDF files in  
/data\n(python_sample_notes.pdf)"] --> A2["Documents\n(1 per page)"]
        A2 --> A3["Text Chunks"]
        A3 --> A4["Chunk  
Embeddings\n(vectors)"]
        A4 --> A5["Chroma Vector  
DB\n/vector_store/chroma_db"]
    end
```

```

    subgraph P2["PHASE 2: QUERY (app.py –
run every time)"]
        B1["User Question"] -->|"embed with
same\nMiniLM model"| B2["Question
Embedding"]
        A5 -->|"load_vector_db()"| B3
        B2 --> B3["retriever.py\nsimilarity
search, k=3"]
        B3 --> B4["Top-3 Relevant Chunks"]
        B4 -->|"join with \n\n"|
B5["Context String"]
        B5 -->
B6["rag_prompt.py\nPromptTemplate\n(context
+ question)"]
        B1 --> B6
        B6 -->|"strict instruction:\nanswer
ONLY from context"| B7["llm.py\nChatOpenAI
gpt-4.1-mini\ntemperature=0"]
        B7 --> B8["Final Answer\nprinted to
console"]
    end

    style A5
fill:#ffe9b3,stroke:#333,stroke-width:2px
    style B7
fill:#c9e7ff,stroke:#333,stroke-width:2px
    style B8
fill:#c9ffd6,stroke:#333,stroke-width:2px

```

Phase 1 — Ingestion (`ingest.py` , run once)

1. `loaders/pdf_loader.py` — `PyPDFLoader` reads PDFs from `data/` , one `LangChain Document` per page.
2. `splitter/text_spiltter.py` — `RecursiveCharacterTextSplitter` breaks pages into 500-char chunks with 100-char overlap, so context isn't lost at chunk boundaries.
3. `embeddings/embedding_model.py` — each chunk is embedded with `sentence-transformers/all-MiniLM-L6-v2` , a free local HuggingFace model (not OpenAI).
4. `vector_store/vector_db.py` — chunks + embeddings are written to a persistent **Chroma** vector DB on disk at `vector_store/chroma_db` .

Phase 2 — Query (`app.py` , run every time)

1. Loads the same embedding model and the existing Chroma DB — no re-ingestion needed.
2. `retriever/retriever.py` wraps Chroma as a retriever doing **similarity search, top-3 (`k=3`)**.
3. User's question is embedded → retriever pulls the 3 most similar chunks → joined into one `context`

string.

4. `prompts/rag_prompt.py` – a strict `PromptTemplate` inserts `context` + `question`, instructing the model to answer *only* from context, else reply "I don't know."
5. `llm/llm.py` – the filled prompt goes to OpenAI `gpt-4.1-mini` (temperature 0, via `ChatOpenAI`), and the answer is returned.

Note

Embeddings for storage (MiniLM, local/free) and the answering LLM (GPT-4.1-mini, OpenAI API) are two separate models – a standard, cost-efficient RAG pattern.

`app.py` currently imports `load_pdf` and `split_documents` but never uses them – dead code, safe to remove.